OPERATOR OVERLOADING

OPERATOR OVERLOADING

- Operator overloading is more or less like function overloading.
- The mechanism to perform special operations using existing operators is known as operator overloading.
- When an operator is overloaded, it does not lose its original meaning, it just gains some additional meaning.
- Any operators studied this far in this chapter can be overloaded. It means that if we want, we can make these operators perform different tasks along with the tasks they actually do.
- In simple word, Operator overloading enables us to manipulate operators.

OPERATOR OVERLOADING

- An operator function is created to achieve operator overloading. General form of operator function is:

```
return-type operator#(arg-list)
{
//operation to be performed
}
```

- Example:

```
void operator << (int v)
{
//operation to be performed
}
```

CAUTIONS

Three things should be kept in my while overloading operator:

- Precedence of operators can not be changed
- The number of operands that an operator takes can not be changed
- Operator overloading can not be done outside of class

OVERLOADING BINARY OPERATOR

- When a operator function overloads a binary operator, the function will have only one parameter. This parameter will receive the right operand and the left operand generates the call to the operator function.

OVERLOADING UNARY OPERATOR

- Overloading Unary Operator is similar to overloading a binary operator except that there is only one operand to deal with here.
- An operator function that overloads a unary operator does not have any parameter and as there is only one operand, this operand generates the function call.

OPERATOR OVERLOADING

Some other operators that can be overloaded are:

- Subscript Operator “ [ ] ”
- Function Invoking Operator “ ( ) ”
- Pointer Operator & and “ * ”
- Dot operator “ . ”
- Arrow Operator “ -> ”
- New Operator “ new ”
- Delete Operator “ delete ”

OPERATOR OVERLOADING

Operators that can not be overloaded:

- Conditional Operator “ ? : ”
- Scope resolution Operator “ :: ”
- Object Size Information Operator “ sizeof()”

OVERLOADING EXAMPLE

```
#include <iostream>
using namespace std;

class Distance {
   private:
      int feet;             // 0 to infinite
      int inches;           // 0 to 12

   public:
      // required constructors
      Distance() {
         feet = 0;
         inches = 0;
      }

      Distance(int f, int i) {
         feet = f;
         inches = i;
      }
      // method to display distance
      void displayDistance() {
         cout << "F: " << feet << " I:" << inches <<endl;
      }

      // overloaded minus (-) operator
      Distance operator- () {
         feet = -feet;
         inches = -inches;
         return Distance(feet, inches);
      }
};

int main() {
   Distance D1(11, 10), D2(-5, 11);
   -D1;                    // apply negation
   D1.displayDistance();   // display D1
   -D2;                    // apply negation
   D2.displayDistance();   // display D2

   return 0;
}
```

ENUMERATORS

ENUMERATORS

- The “C++” language pre-processor offers a method to create symbols which will be replaced by their values during compilation time.
- These symbols behave like constants and you can’t change their value during run-time, so this might be the ideal way do represent weekdays in a way very clear to humans and very handy for computers.

ENUMERATORS

- Example:

```
#define SUNDAY 0
#define MONDAY 1
#define TUESDAY 2
#define WEDNESDAY 3
#define THURSDAY 4
#define FRIDAY 5
#define SATURDAY 6
```

- Look Carefully, the format is

```
#define   SomeString   SomeValue
```

- The SomeString is an arbitrary chosen name like a variable name
- The SomeValue is just a series of values
- The pre-processor will automatically replace each occurrence of the SomeString with the SomeValue,

ENUMERATORS EXAMPLE

The + operator is unaware of weekdays at all and may skip from *SATURDAY* to literally nowhere, leaving the permitted type domain.

adding an *int* to a weekday will take into account the cyclical nature of days and weeks, e.g. *SUNDAY + 15* will result in *MONDAY*

The expression '*day + days*' will invoke our overloaded + operator recursively and the program will crash. We have to explicitly express our will to add two *ints*, not a weekday and an *int*.

```
#include <iostream>

using namespace std;

enum weekday
{SUNDAY,MONDAY,TUESDAY,WEDNESDAY,THURSDAY,FRIDAY,SATURDAY};

weekday operator+(weekday day, int days) {
        return weekday((int(day) + days) % 7);
}

ostream& operator<< (ostream &strm, weekday day) {
        switch(int(day)) {
        case SUNDAY:    strm << "SUNDAY"; break;
        case MONDAY:    strm << "MONDAY"; break;
        case TUESDAY:   strm << "TUESDAY"; break;
        case WEDNESDAY: strm << "WEDNESDAY"; break;
        case THURSDAY:  strm << "THURSDAY"; break;
        case FRIDAY:    strm << "FRIDAY"; break;
        case SATURDAY:  strm << "SATURDAY"; break;
        default:        strm << "Somewhere inside the depths of time..."; break;
        }
        return strm;
}

int main(void) {
        weekday d = SATURDAY;
        d = d + 16;
        cout << d << endl;
        return 0;
}
```

THE END